Estrutura de Dados

Curso Tecnologia em Análise de Desenvolvimento de Sistemas

Aula 05

Prof. Claudio Benossi

1.
Unidade

Vetores, Matrizes, TAD e Alocação Dinâmica de Memória

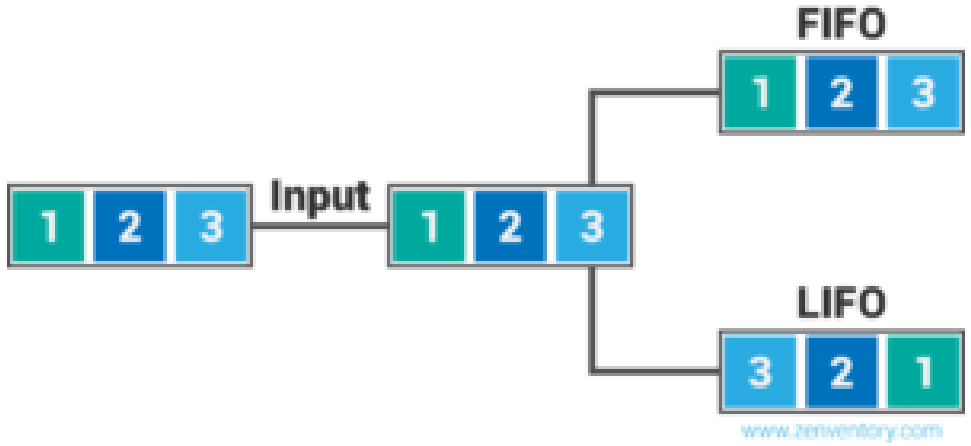
Listas e Pilhas (Stacks)

Agora vamos voltar a nossa atenção a estrutura de dados Fila.

Neste primeiro momento, não vamos tratar aspectos como prioridades, vamos considerar como todos os elementos de forma igual, sendo assim o mesmo comportamento **FIFO** – **F**irs **I**n **F**irst **O**ut (Primeiro a entrar, primeiro a sair).

Listas e Pilhas (Stacks)

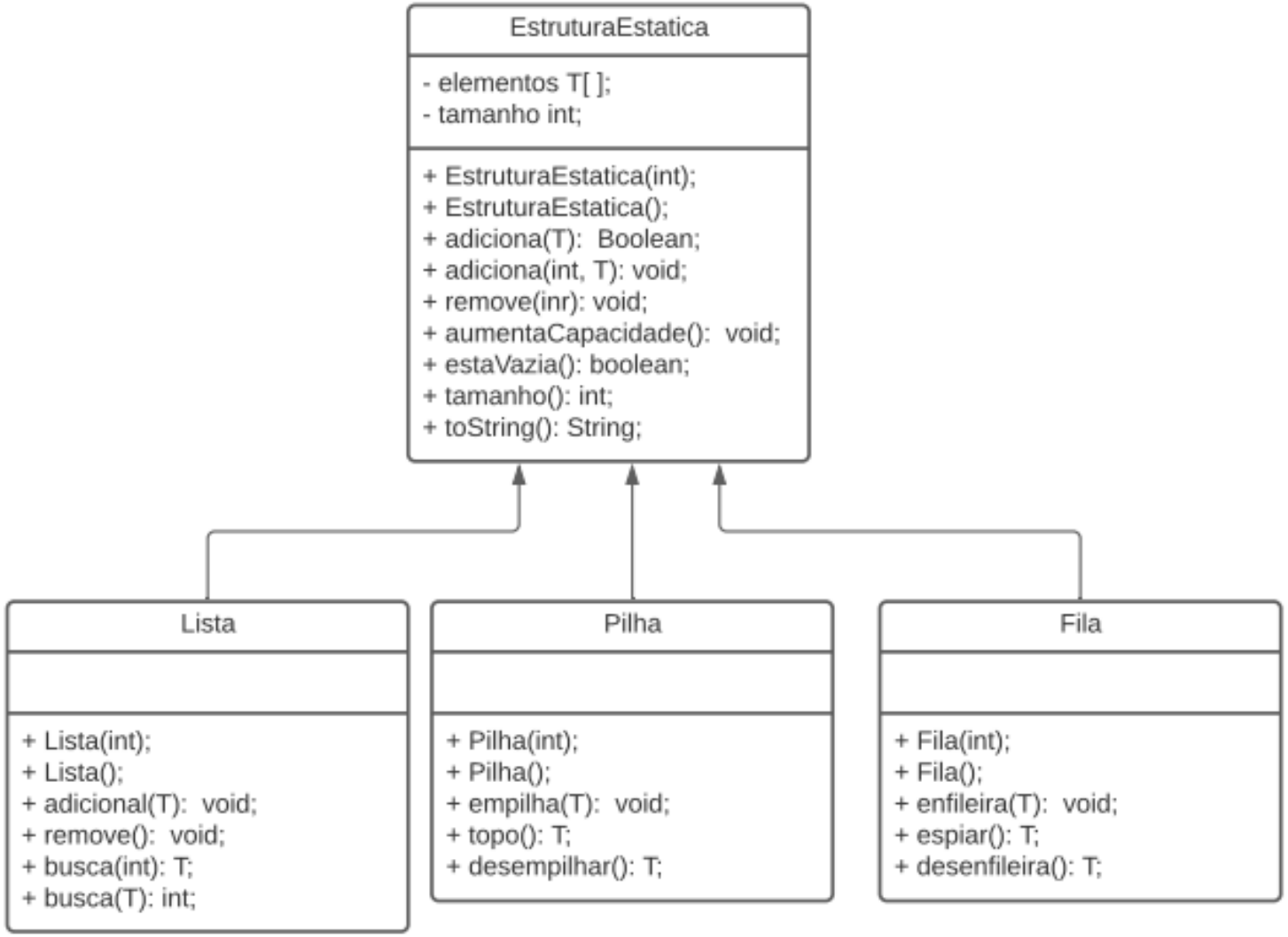
Diferença entre Pilha e Fila



Pilha	Fila
Comportamento: **LIFO** – **L**ast **I**n **F**irst **O**ut (Último a entrar, primeiro a sair)	Comportamento: **FIFO** – **F**irs **I**n **F**irst **O**ut (Primeiro a entrar, primeiro a sair)

Listas e Pilhas (Stacks)

Para facilitar vamos usar nossa estrutura anterior e inserir uma nova classe, conforme o diagrama abaixo:



Listas e Pilhas (Stacks)

Vamos começar criando a classe **Fila**, criando os métodos construtores e o método para enfileirar elementos, seguindo o comportamento da fila **FIFO** – **F**irs **I**n **F**irst **O**ut (Primeiro a entrar, primeiro a sair).

Fila.java

Código-Fonte | Histórico

```java
package estruturaestatica;

public class Fila<T> extends EstruturaEstatica<T> {
    public Fila(){
        super();
    }

    public Fila(int capacidade){
        super(capacidade);
    }

    public void enfileira(T elemento){
        //this.aumentaCapacidade();
        //this.elementos[this.tamanho] = elemento;
        //this.tamanho++;
        // vamos aproveitar o cógido que já existe
        this.adiciona(elemento);
    }

}
```

Listas e Pilhas (Stacks)

Teste:

Fila.java | Teste.java

Código-Fonte | Histórico

```java
1   package estruturaestatica;
2
3   public class Teste {
4
5       public static void main(String[] args){
6           Fila<Integer> fila = new Fila<Integer>();
7           fila.enfileira(1);
8           fila.enfileira(2);
9           fila.enfileira(3);
10          //Vamos verificar se a fila está fazia, nesse caso será false
11          System.out.println("A fila está vazia? " + fila.estaVazia());
12
13          // Vamos verificar o tamanho da minha fila após enfileirar
14          System.out.println("Qual o tamanho da fila: " + fila.tamanho());
15
16          // Vamos visualizar os elementos dentro da fila
17          System.out.println(fila.toString());
18      }
19  }
```

Listas e Pilhas (Stacks)

Resultado:

Saída

Console do Depurador | EstruturaEstatica (run)

```
run:
A fila está vazia? false
Qual o tamanho da fila: 3
[1, 2, 3]
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Listas e Pilhas (Stacks)

Agora que já temos elementos em nossa fila, vamos criar um método para **espiar** a fila, este método não deve alterar a estrutura da fila somente tem como funcionalidade observar e podemos ver que está por exemplo no início da fila:

Fila.java

Source History

```java

    public T espiar(){
        if (this.estaVazia()){
            return null;
        }
        return this.elementos[0];
    }
```

Fila.java | Teste.java

Código-Fonte | Histórico

```java
1   package estruturaestatica;
2
3   public class Teste {
4
5       public static void main(String[] args){
6           Fila<Integer> fila = new Fila<Integer>();
7           fila.enfileira(1);
8           fila.enfileira(2);
9           fila.enfileira(3);
10
11          //Vamos espiar a fila
12          System.out.println(fila.espiar());
13          // poderiamos criar um esturura de decisão com o retorno do método
14          if (fila.espiar() == null){
15              System.out.println("Ninguem na Fila");
16          } else {
17              System.out.println("O elemento da primeira posição é " + fila.espiar());
18          }
19      }
20  }
```

Listas e Pilhas (Stacks)

Resultado:

Saída

Console do Depurador | EstruturaEstatica (run)

```
run:
1
O elemento da primeira posição é 1
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Listas e Pilhas (Stacks)

Agora vamos remover um elemento da fila com o método **desenfileira()**, lembrando do comportamento da fila que é **FIFO** – **F**irs **I**n **F**irst **O**ut (Primeiro a entrar, primeiro a sair).

Fila.java | Teste.java | EstruturaEstatica.java

Código-Fonte | Histórico

```java
1   package estruturaestatica;
2
3   public class Fila<T> extends EstruturaEstatica<T> {
4       public Fila() {...3 linhas }
7
8       public Fila(int capacidade) {...3 linhas }
11
12      public void enfileira(T elemento) {...7 linhas }
19
20      public T espiar() {...6 linhas }
26
27      public T desenfileira(){
28          if (this.estaVazia()){
29              return null;
30          }
31          // ja temos um método para remover na classe pai porém precisa indicar
32          // a posição e para isso vamos criar uma constatnte
33          final int POS = 0;
34
35          T elementoASerRemovido = this.elementos[POS];
36          this.remove(POS);
37          return elementoASerRemovido;
38      }
39
40  }
```

Listas e Pilhas (Stacks)

Teste:

Fila.java | Teste.java

Código-Fonte | Histórico

```java
package estruturaestatica;

public class Teste {

    public static void main(String[] args){
        Fila<Integer> fila = new Fila<Integer>();
        fila.enfileira(1);
        fila.enfileira(2);
        fila.enfileira(3);

        //Vamos visualizar a fila
        System.out.println(fila.toString());

        // Vamos desenfileirar um elemento
        System.out.println(fila.desenfileira());

        //Vamos visualizar a fila novamente apos remover um elemento
        System.out.println(fila.toString());
    }
}
```

Listas e Pilhas (Stacks)

Resultado:

Saída

Console do Depurador | EstruturaEstatica (run)

```
run:
[1, 2, 3]
1
[2, 3]
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Listas e Pilhas (Stacks)

Agora vamos conferi a API do Java que contém todas essas funcionalidades prontas – **API Java Queue**.

Fila.java | Teste.java

Código-Fonte | Histórico

```
1   package estruturaestatica;
2
3   import java.util.Queue;
4
5   public class Teste {
6
7       public static void main(String[] args){
8
9           Queue<Integer> fila = new Queue<Integer>();
10
11      }
12  }
13
14
```

Gerar Métodos p/Implementação

Selecionar métodos para implementar:

- [x] Collection<E>
  - [x] addAll(Collection<? extends E> c) : boolean
  - [x] clear()
  - [x] contains(Object o) : boolean
  - [x] containsAll(Collection<?> c) : boolean
  - [x] isEmpty() : boolean
  - [x] iterator() : Iterator<E>
  - [ ] parallelStream() : Stream<E>
  - [x] remove(Object o) : boolean
  - [x] removeAll(Collection<?> c) : boolean
  - [ ] removeIf(Predicate<? super E> filter) : boolean
  - [x] retainAll(Collection<?> c) : boolean
  - [x] size() : int
  - [ ] spliterator() : Spliterator<E>
  - [ ] stream() : Stream<E>
  - [x] toArray() : Object[]
  - [x] toArray(T[] a) : T[]
- [ ] Iterable<T>
  - [ ] forEach(Consumer<? super T> action)

Selecionar Tudo | Selecionar Nenhum

Gerar | Cancelar

Listas e Pilhas (Stacks)

Você vai perceber que vai dar um erro:

Fila.java | Teste.java

Código-Fonte | Histórico

```
1   package estruturaestatica;
2
3   import java.util.Collection;
4   import java.util.Queue;
5
6   public class Teste {
7
8       public static void main(String[] args){
9
        Queue<Integer> fila = new Queue<Integer>() {
11          @Override
            public int size() {
13              throw new UnsupportedOperationException("Not supported yet.");
            //To change body of generated methods, choose Tools | Templates.
15          }
```

Listas e Pilhas (Stacks)

Para resolver isso, vamos ter um procedimento diferente dos anteriores, porque não podemos criar a classe **Queue()** a partir da API pronta do Java; diferente da classe **stake()** (Pilha) ou a classe **ArrayList()** (Lista) para isso vamos fazer alguns ajustes, e instanciar nossa fila a partir de uma API no caso a **LinkeadList()** (Lista encadeada).

Fila.java | Teste.java

Código-Fonte | Histórico

```java
1   package estruturaestatica;
2
3   import java.util.LinkedList;
4   import java.util.Queue;
5
6   public class Teste {
7
8       public static void main(String[] args){
9           Queue<Integer> fila = new LinkedList<Integer>();
10
11          fila.add(1); // Adiciona
12          fila.add(2);
13          fila.add(3);
14
15          System.out.println(fila.toString()); // Exibe
16
17          System.out.println(fila.peek());
18           // Recupera, mas não remove, o cabeçalho desta fila ou retorna nulo
19           // se a fila estiver vazia ou seja nosso método espiar
20
21          System.out.println(fila.remove());
22           // remove elemento da fila ou desinfileira, por conta da classe Queue
23           // FIFO - Firs In First Out (Primeiro a entrar, primeiro a sair).
24
25      }
26  }
```

Listas e Pilhas (Stacks)

Para encerrar a questão das Filas vamos ver rapidamente a questão da Fila com prioridade, onde a única diferença é você adicionar um elemento em uma posição especifica.

Listas e Pilhas (Stacks)

Nesse caso precisamos fazer uma comparação em relação a prioridade dos elementos para que possamos inserir o elemento no local adequado, usando o método **compare()**;

1.
Unidade

ADO – Atividade Discente Orientada

ADO – Atividade Discente Orientada

- Desenvolver uma aplicação utilizando a linguagem JAVA e os conceitos da Aula 04 e Aula 05, sobre Pilha e Fila e manipulação de dados.
- Vamos desenvolver uma aplicação para controlar a fila de atendimento de um laboratório de coleta de sangue;
- Criar uma Classe Fila com todos os métodos para manipular os dados, inclusive inserir pessoas com prioridade;
- Criar um método para comparar paciente normais de prioridade, e criar uma regra de negócio que para a cada 3 atendimentos preferenciais, 1 normal não havendo preferenciais, segue o atendimento normal.
- Criar uma Classe para testar a aplicação, possibilitando a interação com o usuário, para manipular os dados;

"Todos os nossos sonhos podem-se realizar,
se tivermos a coragem de persegui-los”

Walt Disney

Obrigado!

Se precisar ...

Prof. Claudio Benossi

claudio.benossi@sp.senac.br